

AI Agent

AI Agent 组成概要

推理服务

现状：我们手里的 AI 缺了什么？

第一件装备：Memory（记忆系统）

Memory是什么？

第二件装备：RAG（检索增强生成）

第三件装备：MCP（模型上下文协议）

推荐安装 MCP Server

第四件装备：SKILLS（技能/工具）

更多 SKILLS

github 热门 Skills

1. anthropics/skills
2. OpenSkills (CLI 工具)
3. SkillsMP / SkillsLLM (技能市场)
4. vue3/skills

公司内部前端 SKILLS

在 Trae 中使用 SKILL 开发页面效果

深度思考：

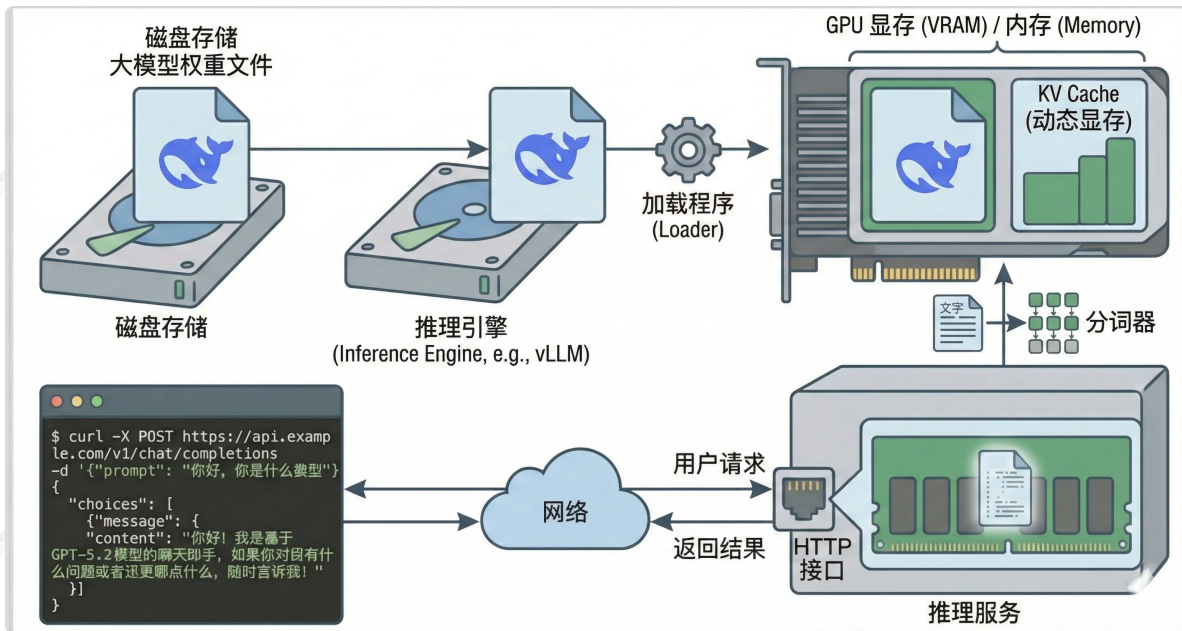
页面效果：

最终形成 AI Agent（智能体）

AI Agent 组成概要

推理服务

像ChatGPT、DeepSeek这类大模型，本质上就是个超大文件，它躺在磁盘上，文件里装的就是训练时学到的知识参数。要让它工作，得有个程序把它加载到内存里，对外暴露HTTP接口，接收用户请求，做推理分析，返回结果，这就是推理服务。



给他配个前端网页聊天框，就成了我们熟悉的聊天AI



现状：我们手里的 AI 缺了什么？

大家现在用的 ChatGPT 或类似工具，本质上是一个 LLM (Large Language Model, 大语言模型)。

它博学多才，像一个刚毕业的高材生，具备极强的逻辑推理能力，但它也有致命弱点：

1. **没记性**：聊完就忘，哪怕你昨天刚告诉它项目代号，今天它又问一遍。
2. **没常识（公司内部）**：它读过互联网上所有开源技术文档，但唯独没读过咱们公司的《技术文档》。
3. **没手脚**：它只能在对话框里打字，不能帮你执行本地 CLI、不能改代码改 Bug、不能操作数据库等。

为了解决这三个问题，需要给这个“大脑”装上**四件套装备**，让他进化成 **AI Agent（智能体）**。也就是当前主流的 Cursor、Trae 等主流 Agent IDE

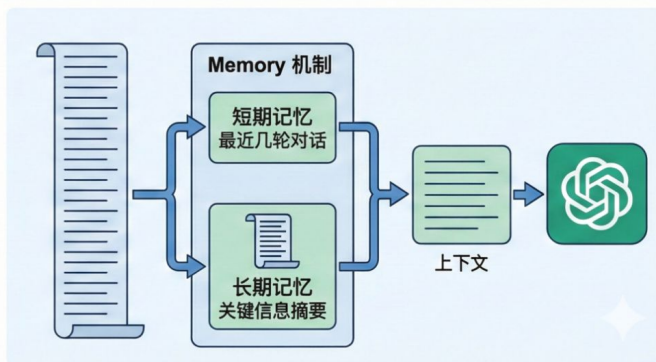
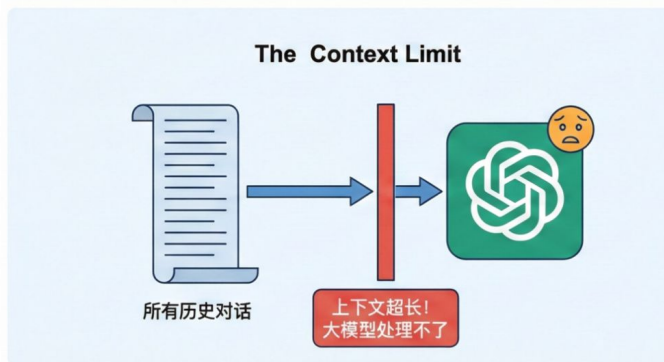
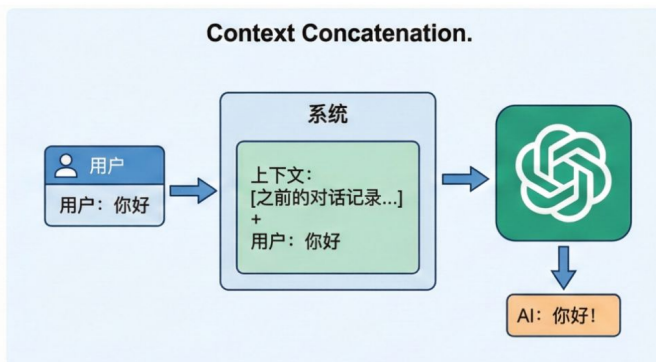
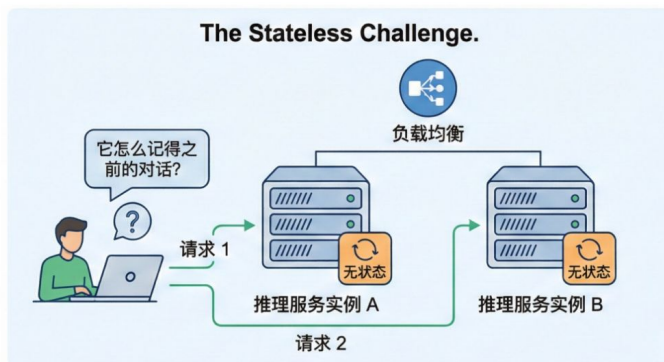
第一件装备：Memory（记忆系统）

Memory是什么？

首先推理服务，本质是个HTTP服务，每个请求进来处理完就结束，本身不保存任何状态。而且为了扛住高并发：一般会部署多个推理服务实例做负载均衡，第一次请求可能打到机器A。第二次请求可能打到机器B，完全是两个不同的进程，

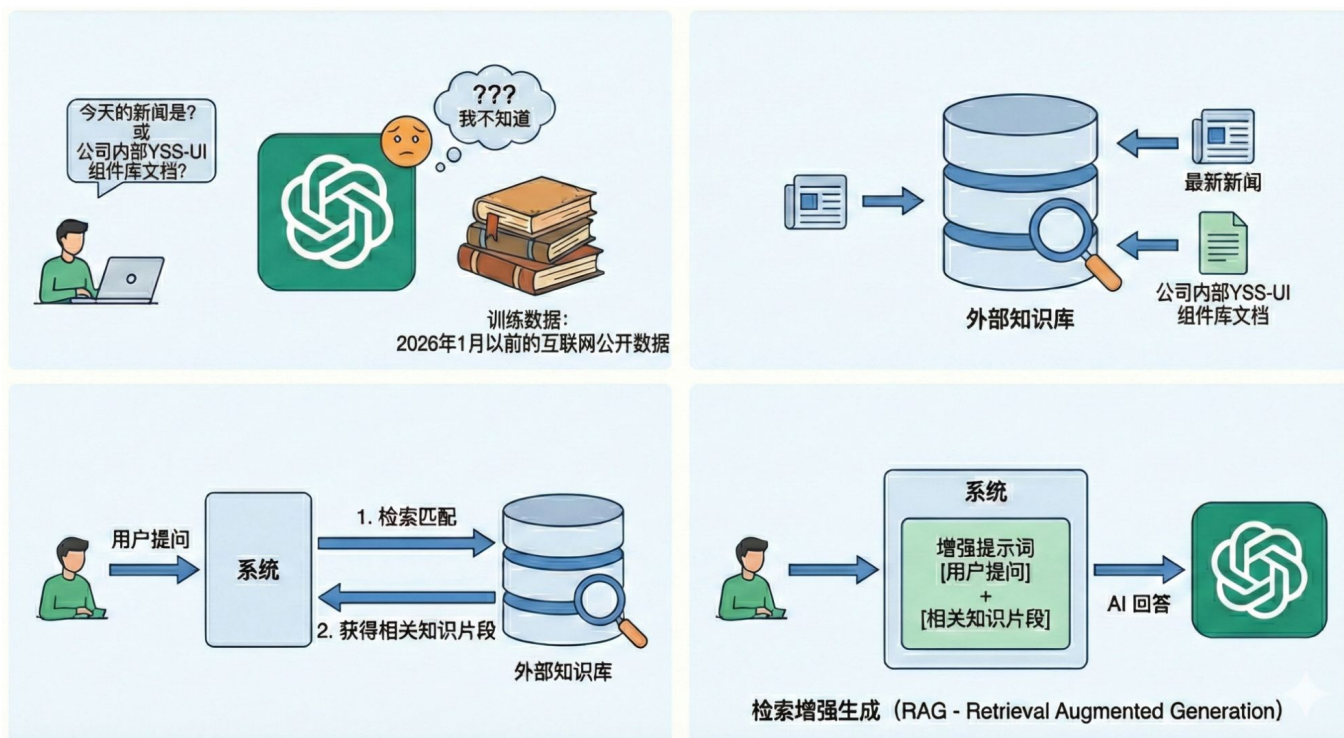
但是，我们在AI聊天页面里明显感觉他能 **记得** 我们之前的对话，这是怎么做到的？其实大模型本身什么都不记得，每次请求时，系统会把之前的聊天记录重新拼到对话里，一起发给大模型，这些拼起来发给大模型的内容统称 **上下文**。

大模型看到完整上下文，自然就能接上话了。但问题又来了，如果每次请求都把所有历史对话发出去，上下文会超长，大模型处理不了，怎么办呢？我们可以分两类管理，当前会话，最近几轮对话完整保存，这叫短期记忆，很久之前的对话，提取关键信息，压缩成摘要，这叫长期记忆。每次请求时都将它们拼成对话发给大模型，这样大模型看起来就像有记忆一样。这套管理上下文的机制就叫 Memory

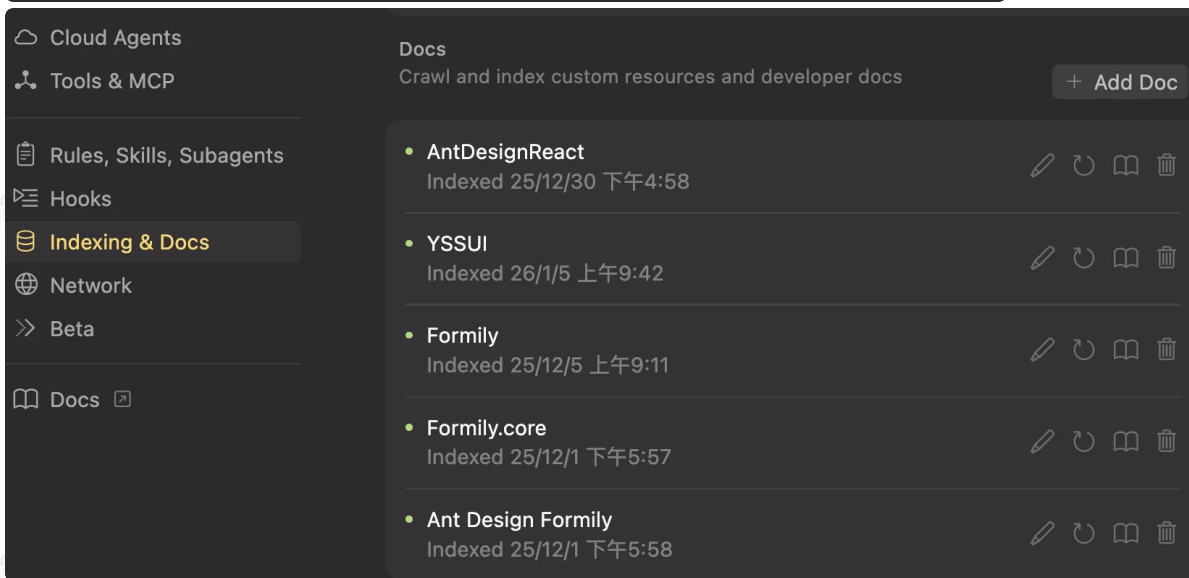


第二件装备：RAG（检索增强生成）

有了记忆，大模型就能记住历史对话了。但新问题来了，大模型的训练数据都是从互联网上抓的历史公开数据，训练后知识固定，你问他今天的新闻或公司内部文档，他根本不知道，怎么办？给他配个外部知识库，放最新新闻、公司内部文档，数据量大就存到数据库里，用户提问时先从数据库做匹配，获得相关知识，再一起喂给大模型。这种检索增强生成 就叫 **retrieval augmented generation**，简称 **RAG**。



像 Cursor / Trae 里的 Docs (文档集) 功能, 本质上就是 RAG 技术的应用。当我们添加 URL 或上传公司内部 Markdown 文档时, 就是在构建一个专属的外部知识库。



使用起来非常简单。当我们需要 AI 基于内部规范写代码时，把刚才索引好的知识库作为‘参考资料’投喂给它。



第三件装备：MCP（模型上下文协议）

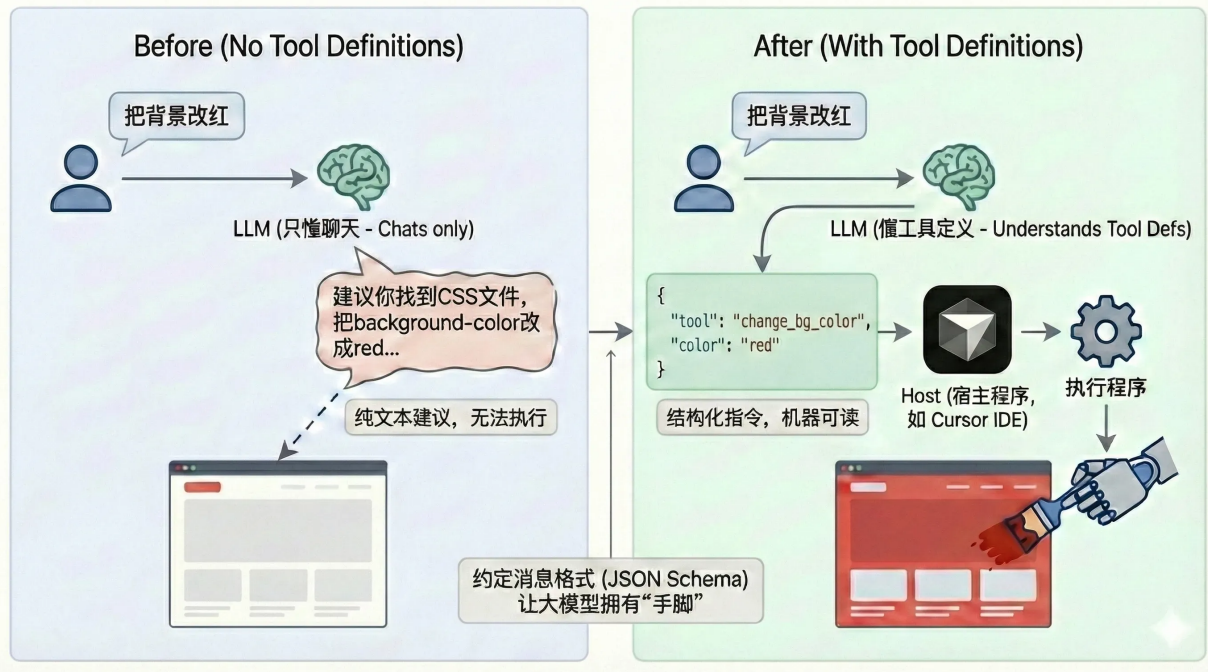
有了Memory和RAG 的加持，大模型能记住历史聊天和获取外部知识了。但新问题又来了，现在大模型只能对话和思考，他没有手脚，怎么让它具备操作工具的能力呢？

1. 核心机制：JSON Schema 契约 我们不再让大模型直接生成代码文本，而是生成一个函数调用请求 (Function Call Request)。

2. 执行流程（以 FileSystem 工具为例）：

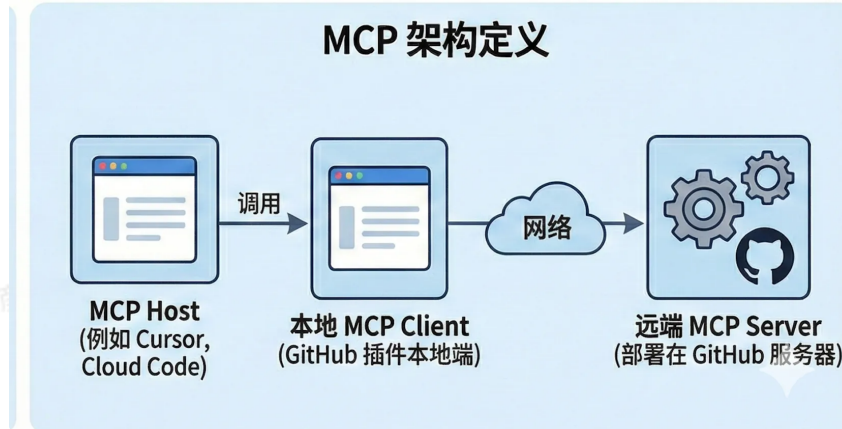
- 预设规范：我们通过 JSON Schema 在系统层面预先定义好接口，告诉大模型：“你有一个 `edit_file` 工具，需要传 `path` 和 `code` 两个参数。”
- 发起请求：当用户指令是“修改 Table 组件背景为红色”时，大模型不再瞎编，而是输出一个结构化的 JSON 数据，指定要调用 `edit_file`，参数包含目标文件 `src/Table.tsx` 和新代码。
- 代理执行：外部系统（Host，如 Cursor）监听到这个请求后，去执行真正的文件写入操作，并将“保存成功”的信号返回给大模型。

3. MCP 架构定义 这种“通过结构化消息驱动外部工具”的标准规范，就是 MCP 协议 (Model Context Protocol)。



在 MCP 架构中, 关键角色分工如下 (如左下图) :

- **MCP Host (宿主/包工头):** 也就是 **Cursor IDE** 或 **Trae**。它负责连接大模型, 分发任务。
- **MCP Server (服务端/干活工):** 真正的执行者, 比如 **GitHub 工具** 或 **FileSystem 工具**。它们负责具体干活 (读写文件、提交代码), 既可以跑在本地, 也可以部署在远程服务器上。



Trae 添加 MCP 配置文档说明 :[Trae IDE - MCP - 更多文档](#)

推荐安装 MCP Server

<https://github.com/modelcontextprotocol/servers>

GitLab 集成: 允许 AI 直接在你的 GitLab 实例上操作, 如查看 Issue、提交代码、创建/审批 Merge Request 等。

figma-dev-mode-mcp-server	Figma 设计对接：允许 AI 读取你的 Figma 设计稿数据（如颜色、间距、图层属性等），帮助它更准确地生成前端代码。
Playwright (浏览器自动化)	让 AI 写完代码后，自动跑一遍浏览器测试，验证页面是否能正常渲染
filesystem (AI IDE 默认配置)	文件系统访问：核心服务，允许 AI 在你指定的目录下进行读写操作
sequential-thinking (AI IDE 默认配置)	逻辑推理增强：这是一种“思维工具”，当 AI 遇到复杂问题时，它会调用这个服务来分步骤、结构化地思考，减少逻辑错误。

```
1 {
2   "mcpServers": {
3     "firebase-mcp-server": {
4       "command": "npx",
5       "args": [
6         "-y",
7         "firebase-tools@latest",
8         "mcp"
9       ],
10      "env": {}
11    },
12    "figma": {
13      "command": "npx",
14      "args": [
15        "-y",
16        "@figma/mcp-server"
17      ],
18      "env": {
19        "FIGMA_API_TOKEN": "请在此处填入您刚才生成的_Token"
20      }
21    },
22    "filesystem": {
23      "command": "npx",
24      "args": [
25        "-y",
26        "@modelcontextprotocol/server-filesystem",
27        "/Users/wangyancong/Documents/",
28        "/Users/wangyancong/Documents/Vue3"
29      ]
30    },
31    "sequential-thinking": {
32      "command": "npx",
33      "args": [
34        "@modelcontextprotocol/server-sequential-thinking"
35      ]
36    },
37    "context7": {
38      "command": "npx",
39      "args": [
40        "-y",
41        "@upstash/context7-mcp"
42      ]
43    },
44    "playwright": {
45      "command": "npx",
```

```

46     "args": [
47         "-y",
48         "@executeautomation/playwright-mcp-server"
49     ]
50 },
51 "gitlab": {
52     "command": "npx",
53     "args": [
54         "-y",
55         "@modelcontextprotocol/server-gitlab"
56     ],
57     "env": {
58         "GITLAB_PERSONAL_ACCESS_TOKEN": "xxxgzKqvkZxxwifSX",
59         "GITLAB_API_URL": "xxx"
60     }
61 }
62 }
63 }

```

第四件装备：SKILLS（技能/工具）

MCP 协议和插件解决了“大模型如何调用工具”的问题，但新问题又来了：插件那么多，大模型怎么知道面对一个复杂任务时，该先用哪个、后用哪个？怎么组合最稳妥？

这就好比给了一个实习生 VS Code 插件和调试工具，他也不一定能修好一个复杂的 Bug。他缺的不是工具，而是工程师的经验和排查思路。

那好办，我们把这些“套路”写成一份结构化的操作手册（SOP），详细规定：遇到什么场景，先做什么检查，再做什么操作，最后怎么验证。

这份“结构化的操作经验”，在 Agent 体系里就叫 Skills。

更多 SKILLS 在文章最下面介绍 📌

更多 SKILLS

github 热门 Skills

1. anthropics/skills

Anthropic 官方推出的 Agent Skills 框架的公开技能标准 + 示例集合仓库

GitHub – anthropics/skills: Public repository for Agent Skills

2. OpenSkills (CLI 工具)

🔗 openskills/AGENTS.md at main · numman-ali/openskills

从任意 GitHub 仓库下载符合 `SKILL.md` 格式的技能包，并注入到 AI Agent 的上下文中

此处为语雀内容卡片，点击链接查看：

https://www.yuque.com/sunluyong/survivor/vkv5s0alu2d91sct?view=doc_embed

3. SkillsMP / SkillsLLM (技能市场)

聚合了 15 万+ 个开源 Skills 的搜索平台



Just a moment...

<https://skillsmp.com/zh>

4. vue3/skills

🔗 GitHub – vuejs-ai/skills: Agent skills for Vue 3 development

公司内部前端 SKILLS

<http://192.168.164.27:3200/skills>

代码仓库地址：<http://192.168.167.142:8081/Data-Middleground-Develop-Area/product-code/web/vue/basic-component/yss-ui/-/tree/dev/packages/skills>

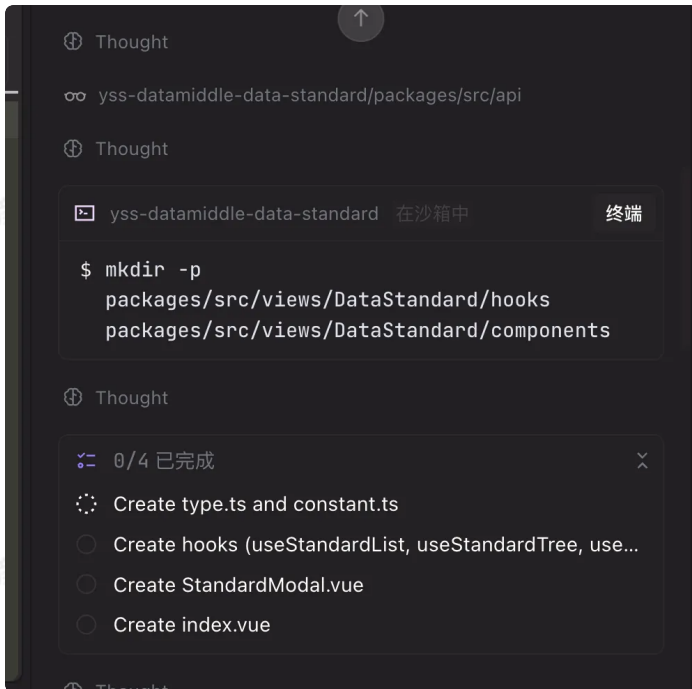
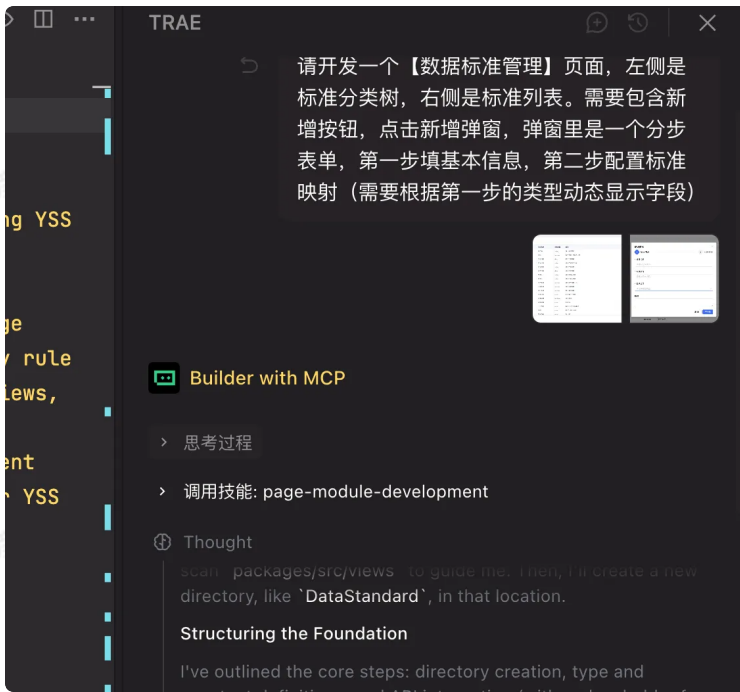
标准化生成“增删改查”页面 Skill 示例	自动创建目录 -> 生成 <code>type.ts</code> -> 生成 API 文件 -> 生成 Vue 组件 -> 生成 Hooks。保证目录结构统一
API 集成 Skill	规范前后端接口调用的写法 读取 Swagger/YApi 定义 -> 生成符合项目 axios 封装规范的请求代码

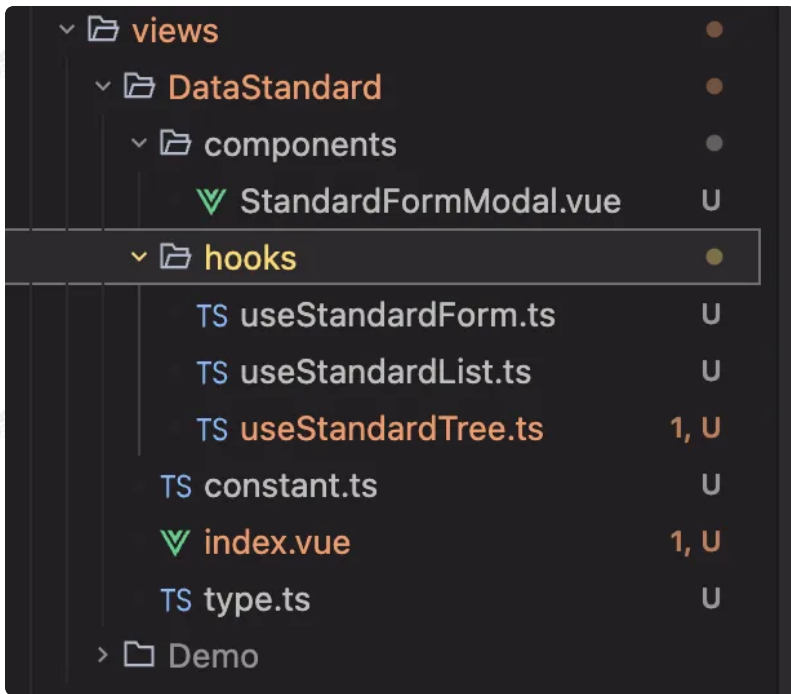
Hooks 使用 Skill	强制 AI 复用组件库开发已有的 Hooks 在写组件逻辑前，先检索 <code>@yss/hooks</code> 目录，优先复用 <code>useTableHeght</code> 、 <code>useTreeHeght</code> 等逻辑，而不是自己手写一遍
项目提交规范 Skill	提交前自动运行 Lint 检查 -> 检查 Commit Message 格式 -> 阻止不合规的代码入库

在 Trae 中使用 SKILL 开发页面效果

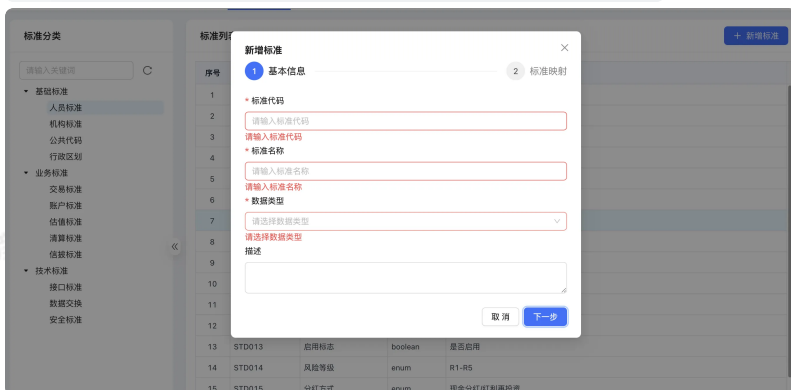
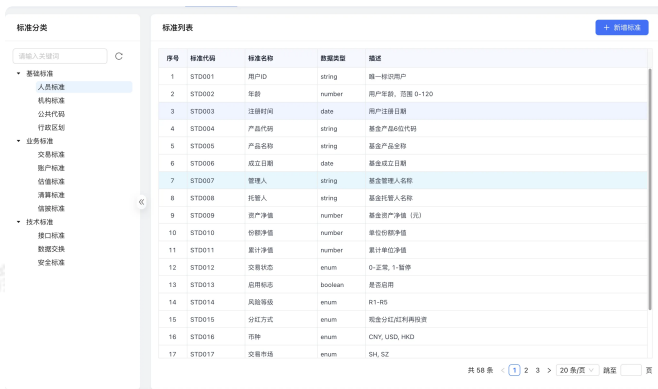
请开发一个【数据标准管理】页面，左侧是标准分类树，右侧是标准列表。需要包含新增按钮，点击新增弹窗，弹窗里是一个分步表单，第一步填基本信息，第二步配置标准映射（需要根据第一步的类型动态显示字段）

深度思考：





页面效果：



最终形成 AI Agent（智能体）

当我们把上面四个模块组装起来，就是我们像 Trae、Cursor 等主流的 Agent IDE；

🧠 大脑 (LLM) > + 📝 记事本 (Memory)+ 📖 公司资料 (RAG)+ 🚀 万能接口 (MCP)+ 🛠️ 专业招式 (Skills)= AI Agent (员工)

